

紙芝居アプリ ソフトウェア開発者向け資料

目次

1. 管理範囲と責務を理解する	2
2. 開発環境を準備する	3
2.1 必要な環境	3
3. リポジトリ構成を把握する	4
4. 共通の開発フローに従う	5
5. アプリSB3を変更する	5
6. 埋め込み機能拡張を更新する	6
7. ビルダーを変更する	7
8. ドキュメントとサイトを変更する	7
9. 関連プロジェクトを確認する	8
9.1 sb3-toolchain	8
9.2 Viteプラグイン	8
9.3 TurboWarp 機能拡張開発用テンプレート	8
9.4 TurboWarp 機能拡張	8
9.5 その他のライブラリ	9
10. 関連ドキュメントを確認する	10

紙芝居アプリ ソフトウェア開発者向け資料

Copyright © 2026 Hiroya Kubo. この文書は [CC BY-SA 4.0](#) で提供します。

このガイドは、TMPose紙芝居のアプリ、SB3ソース、ビルダー、Webサイト、ドキュメントを 変更する 開発者向けの入口です。アプリの内部構造、成果物、ビルダー仕様、検証、公開は [紙芝居アプリ内部仕様書](#)、台本の書式とコマンド仕様は [台本DSLマニュアル](#)と[コマンドリファレンス](#) を正本とします。

対象アプリ/DSL: `kamishibai=3.1`

過去のバージョンからの変更は [history.md](#) を参照してください。

このガイドの章は、次の3つの区分で並んでいます。

区分	対象	読み方
導入	管理範囲の理解から共通の開発フローまで	初めて開発するときに、この順に読む
変更対象別手順	アプリSB3、機能拡張、ビルダー、文書とサイト	変更対象に応じて、必要な章だけを読む
参照	関連プロジェクトと関連ドキュメント	関連プロジェクトや資料を探すときに参照する

「導入」では、管理範囲を確認し、開発環境とリポジトリを把握してから、共通の開発フローへ進みます。「変更対象別手順」は、この順に実施する一連の工程ではなく、変更対象ごとに独立した手順です。「参照」は作業手順ではありません。

1. 管理範囲と責務を理解する

このリポジトリが管理するものは次のとおりです。

- 物語固有の台本やアセットを含まない、汎用の紙芝居アプリ
- アプリSB3の展開ソースと、配布用SB3を生成する設定
- 台本と画像・音声をSB3へ組み込むnpmパッケージ
- GitHub Pagesへ公開するWebサイトと一般向けドキュメント
- 上記を検証する自動テスト

関連プロジェクトとの境界は次のとおりです。

対象	管理場所	このリポジトリとの関係
SB3の展開・検証・決定的再構築	kubohiroya/sb3-toolchain	固定依存として利用する。「 sb3-toolchain 」(9.1 節) を参照
浦島太郎などの公開用物語	kubohiroya/tmpose-kamishibai-samples	stories/urashima/ で台本、固有アセット、生成物を管理する
埋め込み機能拡張	各機能拡張のGitHub リポジトリ	app/ には検証済み成果物と由来情報だけを同期する
TurboWarp Extension Gallery の機能拡張	Gallery の公開URL	SB3 から外部URL を参照する

公開サンプル の固有ファイルを本体へコピーしません。本体の汎用性と、サンプルの独立した更新・配布を維持します。

依存バージョン、スクリプト、公開対象の正本は `package.json` と `pnpm-lock.yaml` です。文書には特定 commit を転記せず、必要なときに確認します。

```
pnpm why @kubohiroya/sb3-toolchain
git diff -- package.json pnpm-lock.yaml
```

2. 開発環境を準備する

2.1 必要な環境

- Node.js 22.12.0 以上
- pnpm 11
- PDF 生成に利用できる Chrome または Chromium
- Git と、GitHub 操作に利用する GitHub CLI

```
corepack enable
pnpm install
```

CI では lockfile 以外の依存解決を許可しません。

```
pnpm install --frozen-lockfile
```

初回セットアップ後は、展開SB3ソースとテスト用SB3を確認します。

```
pnpm sb3:check
pnpm test
```

macOS では通常の Google Chrome を PDF 生成に自動利用します。別のブラウザを使う環境では、`VIVLIOSTYLE_CHROME_PATH` へ実行ファイルの絶対パスを設定します。

3. リポジトリ構成を把握する

パス	役割
<code>app/</code>	紙芝居アプリ SB3 の Git 管理上の正本
<code>app/project.source.json</code>	整形済み Scratch プロジェクト
<code>app/assets/</code>	汎用アプリ自身が使用する画像・音声
<code>app/extensions/</code>	埋め込み機能拡張の同期済み JavaScript
<code>app/embedded-extensions.json</code>	埋め込み機能拡張の管理情報
<code>app/sb3-source.json</code>	SB3 展開ソースのマニフェスト
<code>src/builder/</code>	npm で公開する SB3・台本変換 API
<code>bin/</code>	npm CLI のエントリーポイント
<code>scripts/</code>	サイト、ドキュメント、配布物のビルドと検証
<code>docs/general/</code>	一般向け・開発者向けの文書原稿
<code>docs/workshops/</code>	日付付き体験会資料
<code>site/</code>	GitHub Pages の静的入力
<code>test/</code>	ビルダー、SB3、VM、ドキュメント、公開契約のテスト

次の場所は生成物であり、Git 管理上の正本ではありません。

パス	内容
<code>tmp/kamishibai.sb3</code>	TurboWarp 編集と自動テストに使う SB3
<code>dist/</code>	GitHub Pages へ公開するサイト、HTML/PDF、配布用 SB3
<code>output/pdf/</code>	印刷用 PDF のローカル確認先

4. 共通の開発フローに従う

すべての変更は GitHub Issue へ受け入れ基準とロールバック手順を記録し、小さなブランチと PR へ分けま

す。

1. 最新の `main` から作業ブランチを作る。
2. `pnpm install --frozen-lockfile` で依存を復元する。
3. 変更対象に近いテストを先に実行する。
4. 生成物ではなく正本を変更する。
5. 差分と生成結果を確認する。
6. 標準チェックと必要な手動確認を行う。
7. Issue の運用ログと DoD を更新して PR を作成する。

無関係な変更や未追跡ファイルをまとめてコミットしません。SB3 の `import` や成果物の 置換を行う前には、必ず `git status` と対象パスの差分を確認します。変更対象ごとの テストと公開前チェックは 内部仕様書の検証 を 参照してください。

5. アプリ SB3 を変更する

このリポジトリでは `app/` をアプリ SB3 の正本とし、固定した `sb3-toolchain` を `pnpm sb3:*` スクリプトから利用します。展開ソース形式、`import` と `build` の上書き保護、 決定的出力の共通仕様は `sb3-toolchain` の SB3 ソース管理ワークフロー を参照してください。

`app/` から編集用 SB3 を生成します。

```
pnpm sb3:build
```

`tmp/kamishibai.sb3` を TurboWarp で編集し、別の明示的なパスへ保存してから取り込みます。

```
pnpm sb3:import -- /path/to/edited-kamishibai.sb3
git diff -- app
pnpm sb3:check
pnpm test
pnpm run build
```

`app/project.source.json` は次の不変条件を維持します。

- 浦島太郎などの公開サンプル固有ターゲット、画像、音声を含めない
- 台本解析・実行用リストを空の初期状態で保持する
- 組み込み台本用の予約変数を一意に保持する

展開形式とアセット・拡張の整合性は `pnpm sb3:check` で検証します。toolchainの共通 検証項目を本ガイドへ重複して列挙しません。

DSL、Loading表示、入力、分岐、テキスト、画面遷移などの振る舞いを変更するときは、同じPRで [DSL資料](#)、[コマンド資料](#)、[内部仕様書](#)、VMまたはブロック構造のテストを更新します。

6. 埋め込み機能拡張を更新する

`app/extensions/` のJavaScriptは同期済み成果物です。バグ修正や機能追加は、`app/embedded-extensions.json` に記録された上流リポジトリで行い、レビュー済みの 成果物だけを本リポジトリへ取り込みます。

`status`、`sync`、`update` の意味、由来情報の形式、transactionalな更新、ID移行の 対象schemaは、`sb3-toolchain` の [SB3ソース管理ワークフロー](#) と [埋め込み拡張IDの移行](#) を正本とします。ここでは紙芝居アプリへ反映する手順だけを示します。

```
pnpm sb3:extensions:status
pnpm sb3:extensions:sync
pnpm sb3:extensions:update -- EXTENSION_ID
```

上流で拡張IDが変更された場合は、toolchainのID移行を伴う更新を使います。

```
pnpm sb3:extensions:update -- OLD_ID --migrate-id NEW_ID
```

上流の成果物パスも変わった場合だけ `--artifact PATH` を追加します。更新後は必ず `git diff -- app`、`pnpm sb3:check`、`pnpm test`、`pnpm run build` を確認し、生成したSB3をTurboWarpで開いて対象拡張の主要機能を確認します。

7. ビルダーを変更する

公開APIは `src/builder/index.js`、CLIは `src/builder/cli.js` と `bin/`、仕様テストは `test/builder.test.mjs` にあります。公開API、CLI、アセットマニフェスト、決定的生成、transactional更新の現行仕様は 内部仕様書のSB3・台本変換ビルダー を参照してください。

APIまたはCLIを変更するときは次を同じ変更を含めます。

- API入力、返り値、エラーの後方互換性の判断
- CLIの `--help` と引数検証
- アセットマニフェストと出力manifestの形式
- 決定的生成とtransactional更新のテスト
- 内部仕様書のAPI/CLI例
- 破壊的変更の場合は新しいメジャーバージョン

```
node --test test/builder.test.mjs
node bin/tmpose-kamishibai.mjs --help
pnpm typecheck
pnpm pack:check
```

8. ドキュメントとサイトを変更する

一般文書は `docs/general/`、体験会資料は `docs/workshops/<日付>/`、公開入口は `site/` を 正本とします。

```
pnpm run preview:docs
pnpm run preview:workshop
pnpm run preview:staff
```

子供向け概要書と参加者向け体験会資料だけに `rubygana` を適用します。確認する学年を 変更する場合は1から6を指定します。

```
RUBYGANA_GRADE=4 pnpm run build
```

`pnpm run build` は一般文書ごとにWeb Publicationを構築し、Vivliostyle CLIの `toc` 設定で `h2`・`h3` までを含む目次を生成します。目次をMarkdownへ重複して記述しません。HTML、Vivliostyle Viewer、PDF、文書横断目次、画像参照、しおり、favicon、ライセンス、配布SB3をまとめて検証します。Markdownだけを確認して完了にせず、生成されたHTML/PDFも確認します。

Markdownの見出しには章・節番号を書きません。h1 は番号なしの文書名、h2 と h3 は 本文と目次で自動採番します。用語集などの前付けを採番しない場合は、見出しへ `{.unnumbered}` を付けます。本文から見出しを参照するときは、番号ではなく意味の変わらない ID を付けてリンクし、組版結果にも現在の章・節番号を表示する場合は、リンクへ `{data-ref="chapter"}` または `{data-ref="section"}` を付けます。

9. 関連プロジェクトを確認する

TMPose 紙芝居の開発から分離し、他の TurboWarp 作品や開発環境でも利用できるものを 各リポジトリで公開しています。各プロジェクトの仕様、開発手順、リリースはリンク先を 正本とします。

9.1 sb3-toolchain

`sb3-toolchain` は、SB3 を Git 差分可能な 展開ソースとして管理し、検証して決定的に再構築するための CLI / JavaScript API です。このリポジトリでは固定依存として利用し、`app/` の import、検証、build、埋め込み 機能拡張の同期と ID 移行を担います。

- [SB3 ソース管理ワークフロー](#)
- [SB3 展開ソース形式 v1](#)
- [埋め込み拡張 ID の移行](#)

9.2 Vite プラグイン

- `vite-plugin-turbowarp-extension`: TypeScript プロジェクトを単一ファイルの TurboWarp 機能拡張として build する Vite プラグイン

9.3 TurboWarp 機能拡張開発用テンプレート

- `turbowarp-extension-template`: Vite を使った TurboWarp 機能拡張の開発、テスト、build、リリース用テンプレート

9.4 TurboWarp 機能拡張

- `turbowarp-tpose`: Teachable Machine Pose モデルを利用したカメラ姿勢認識
- `turbowarp-text-lines`: テキストの行数取得、行単位の読み出し・分割
- `turbowarp-asset-manager`: IndexedDB と SB3 内の画像・音声を扱うアセット管理
- `turbowarp-async-input`: キーボード、ポインター、姿勢入力を対象ごとに扱う非同期入力
- `turbowarp-runtime-expression`: runtime 変数を使う条件式の安全な評価と broadcast 監視

9.5 その他のライブラリ

- rubygana : 日本語テキストの読み仮名を生成する Node.js ライブラリ

10. 関連ドキュメントを確認する

- [03-user-guide.md](#): アプリの利用方法と成果物の使い分け
- [04-dsl-manual.md](#): 台本の構造と書き方
- [05-command-reference.md](#): コマンドとアクションの仕様
- [07-internal-specification.md](#): アプリ内部構造、成果物、検証、公開
- [history.md](#): DSL とアプリの変更履歴
- [README.md](#): プロジェクト全体の入口と主要コマンド